

第 18 章 | 強化學習：AI 的互動與試錯

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線

在機器學習的體系中，強化學習（Reinforcement Learning, RL）處理序列決策問題。不同於監督式學習依賴人類專家標註的「靜態解答」，或非監督式學習專注於挖掘資料內部的「靜態結構」，強化學習的核心在於互動驅動（Interaction-driven）。

這是一種模擬生物學習本能的架構：代理人（Agent）被置於一個未知的環境中，沒有人告訴它每一步該怎麼做，它必須透過不斷的「試錯」（Trial and Error），從環境回傳的獎勵訊號中累積經驗。這種從動態互動中學習決策序列的能力，可用於機器人控制、自動駕駛與策略遊戲等序列決策問題。

本章學習目標

掌握 MDP：理解馬可夫決策過程的數學模型與「未來僅取決於當下」的假設。

理解探索與利用：說明如何平衡「嘗試未知動作」與「採用目前較佳動作」。

區分 MC 與 TD：辨析「事後檢討」與「邊做邊改」兩大演算法流派的實務優劣。

實作 Q-Learning：運用表格型演算法解決離散狀態空間的決策問題。

佈署 DQN：結合深度神經網路處理高維度或連續狀態的實戰挑戰。

學習先備知識

開始本章前，請確認你具備以下基礎：

神經網路基礎：理解多層感知器、梯度更新（Gradient Updates）與損失函數。

Keras 操作經驗：能夠熟練使用 `Sequential` 模型建構網路架構。

機率與期望值概念：理解隨機變數、期望值（Expectation）與機率分佈的意義。

機器學習體系對照表

學習類別	標籤依賴	目標設定	學習來源
監督式學習	極高（依賴專家標註答案）	最小化成本函數（誤差）	靜態資料集
非監督式學習	無（處理未標註資料）	提取資料特徵或分群	靜態資料集
強化學習	無（僅有獎勵訊號）	最大化長期累積獎勵	與環境動態互動

在理解了強化學習「互動勝於觀察」的定位後，我們必須先建立其核心的世界觀，也就是代理人與環境之間如何構成一個周而復始的決策迴圈。

2. 代理人與環境的互動迴圈：MDP

強化學習的所有決策都建立在一個簡單的直覺上：觀察 → 行動 → 回饋。這就像我們教電腦打電動，或者訓練小狗坐下，本質上都是建立一套互動機制。

核心觀念：代理人與環境

代理人 (Agent)：決策的核心 (AI 的大腦)，負責觀察局勢並決定動作。

環境 (Environment)：遊戲規則的制定者，決定代理人的動作會導致什麼後果。

當代理人在「環境」中執行一個「動作」時，環境會吐出兩個資訊：一是新的局面（狀態），二是代理人表現得好不好的獎勵。代理人的學習目標，就是透過無數次的循環，摸索出能拿到最高總分的方法。

重要術語中英對照

Agent（代理人）：進行學習與決策的主體。

Environment（環境）：Agent 以外的所有世界，負責狀態轉移與發放獎勵。

State (St)：代理人在時間點 t 觀察到的環境快照（如遊戲畫面、角度）。

Action (At)：代理人在特定狀態下採取的行動（如向左、向右）。

Reward (Rt)：環境在動作後給予的即時回饋標量值（可正可負）。

馬可夫決策過程 (MDP)

為了將上述互動數學化，我們使用 MDP 模型。MDP 的核心假設是：未來僅取決於當下。這意味著只要現在的資訊足夠完整（目前的狀態 St ），我們就不需要去翻閱過去所有的歷史紀錄來做決策。這種簡化大大降低了對複雜現實問題建模的難度，讓 AI 能專注於處理當下的判斷。

雖然互動是為了累積獎勵，但 AI 面臨的第一個挑戰，是當它發現一個能得分的方法時，該「見好就收」還是「冒險嘗試」。

3. 探索 (Exploration) 與利用 (Exploitation) 的權衡

在實務開發中，「探索與利用」的平衡決定了 AI 能達到什麼樣的高度。這是一種策略性的兩難困境。

兩難困境解析

想像一個剛畢業的大學生：

利用 (Exploitation)：選擇立刻去一家已知薪水不錯的公司上班。這能確保短期穩定收益，但可能會錯過更有潛力的領域。

探索 (Exploration)：嘗試未知的專業或進修。雖然短期內沒有收入甚至有風險，但長期而言可能發現能賺 100 倍的工作。

如果 AI 過度「利用」，它會卡在「區域最佳解」，例如在迷宮中只敢往右走拿 1 分，而永遠不會發現往左走其實有加 100 分的關鍵寶箱。

epsilon\greedy 策略與衰減機制

為了解決困境，我們引入 epsilon\greedy 策略：

以小機率 epsilon 讓 AI 執行「隨機亂走」（探索）。

以大機率 $1 - \text{epsilon}$ 讓 AI 執行「目前已知最強動作」（利用）。在程式實作中，我們會使用 `epsilon_decay`：訓練初期使用較大的 epsilon 增加探索；之後逐步降低 epsilon，提高採用目前較佳動作的比例。

折扣因子 (γ)

為什麼「現在的 100 元」比「未來的 100 元」更重要？這是因為未來充滿變數（如同通貨膨脹）。在 RL 中， γ （0 到 1 之間）決定了 AI 對未來回報的重視程度：

γ 接近 0：AI 極度短視，只在乎下一步拿多少分。

γ 接近 1：AI 具備遠見，願意為了長遠目標忍受暫時的分數停滯。此外， γ 也能防止 AI 在沒有終點的環境中無限繞圈，逼迫它尋求效率。

有了決策策略後，AI 需要一套「衡量標準」來判斷目前的局面到底是「真好」還是「看起來好」。

4. 局勢的衡量尺：策略與價值函數

「價值函數」是將長期回報量化的工具，它賦予了 AI 佈局未來的能力。

關鍵術語區分：Reward vs. Value

Reward（短期回饋）：下棋時吃掉一顆子立刻得到的即時加分。

Value（長期價值）：評估目前棋盤上的陣勢，即使當下沒吃子，但此局面未來贏球機率極高，因此「價值」很高。

價值函數定義

狀態價值函數 $v_{\pi}(s)$ ：評估「待在這個狀態」預期能拿多少分。

動作價值函數 $q_{\pi}(s,a)$ ：評估「在此狀態採取某動作」預期能拿多少分。

貝爾曼方程式 (Bellman Equation)

這是動作價值更新公式，其核心邏輯非常直覺：當前價值的預估 = 即時獎勵 + ($\gamma \times$ 下一步的預估價值) 透過這個遞迴邏輯，未來的回報會像漣漪一樣傳導回當下的決策，讓 AI 在沒有標準答案的情況下學會預判情勢。

建立了評估標準後，下一步是決定「何時」更新這些預估值，這引出了 RL 兩大演算法流派。

5. 學習時機的辯證：蒙地卡羅 (MC) vs. 時序差分 (TD)

蒙地卡羅 (MC)：「期末考」模式

MC 必須等到一個回合徹底結束（例如完成一個 episode 或進入終止狀態），才開始回頭結算總分並修正價值預估。這種方法穩定但非常緩慢，因為 AI 必須等待漫長的 Episode 結束才能學到一次經驗。

時序差分 (TD)：「隨堂小考」模式

TD 採用拔靴法 (Bootstrapping)。它不需要等遊戲結束，每走一步，就利用「下一步的預估值」來更新「這一步的價值」。這種「邊走邊學」的特性使其收斂速度通常大幅優於 MC。

觀念釐清：估計值如何逐步變準

你可能會問：「用『預估值』更新『預估值』，不是瞎猜嗎？」觀念說明：雖然初期是瞎猜，但只要 AI 在萬次嘗試中偶爾碰到了「真實獎勵」（如獲勝的 +1），這個真實訊息就會透過 TD 邏輯，穩穩地一步步往回傳遞，最終讓整張價值表趨於精準。

理論準備就緒，接下來進入實作環節，介紹 AI 界的共通語言——OpenAI Gym。

6. 實作基石：OpenAI Gym 環境導讀

OpenAI Gym 提供標準化的介面，讓開發者無需自創物理引擎。它是連結理論與程式碼的橋樑。

核心指令集

`gym.make('CartPole-v1')`：載入倒立擺遊戲環境。

`env.reset()`：重置環境，開啟新局並取得初始狀態 `S0`。

`env.step(action)`：執行動作，是與環境互動的核心。

`env.step(action)` 回傳值拆解

執行後會回傳一個包含四個元素的元組 (Tuple)：

元素	說明	關鍵角色
<code>next_state</code>	執行動作後的環境觀測值	作為下一次決策的輸入
<code>reward</code>	該步獲得的即時分數	學習的指標訊號
<code>done</code>	布林值 (True/False)	控制迴圈的核心：若為 True 必須結束並 <code>reset</code>
<code>info</code>	輔助診斷資訊	訓練時通常略過

實作警示與 CartPole 規範

CartPole 終止條件：pole 角度超過 12 度 或 推車偏離中心超過 2.4 單位 即視為失敗 (`done=True`)。

Colab 環境警示：在雲端執行時，禁止使用 `env.render()`。這會導致伺服器崩潰，請改用數值監控或虛擬顯示套件。

掌握環境後，我們先從「表格記憶」式的 Q-Learning 開始，學習如何破解方格世界。

7. 表格式方法：Q-Learning 與方格世界

當狀態空間有限時，Q-Learning 是最高效的工具。

Q-table 運作邏輯

在 5x6 的方格世界 (GridWorld, 共 30 個狀態) 中, AI 使用一張二維表格記錄「每個格子」對應「四個動作」的分數。

視覺化提示: 黃色三角形代表 關鍵 (+1), 黑色圓形代表 陷阱 (-1)。

參數解析: 學習率 (α) 決定了 AI 換腦袋的速度。 α 越高, 新經驗覆蓋舊知識的比例越大。

程式碼流程導讀

在 `_learn` 方法中, 核心邏輯是: `self.q_table[s][a] += self.lr * (q_target - q_val)` 其中 `q_target` 是利用 `np.max(self.q_table[next_s])` 取得下一狀態的最優預估, 這就是將「未來的希望」傳給「過去的動作」。

雖然查表法很強, 但當狀態變得很複雜 (如畫面的百萬像素) 時, 就會面臨「維度災難」(Curse of Dimensionality), 表格會因過大而崩潰。這時可改用深度學習近似價值函數。

8. 複雜局勢的救星: 深度 Q 網路 (DQN)

當面臨維度災難時, 神經網路是必然選擇。DQN 本質上就是將貝爾曼方程式落實於網路架構的「Neural Bellman」實作。

核心機制創新

從 Table 到 Network: 用 Keras `Sequential` 模型取代 Q-table。關鍵規則: 神經網路的輸出層神經元數量必須等於動作數量, 每個神經元代表該動作的預估 Q 值。

經驗回放 (Replay Memory): AI 將經驗存入 `deque` 容器。使用 `deque` 是因為它能自動處理 `maxlen` (舊記憶自動彈出), 且在存取速度上遠快於標準 `list.pop(0)`。訓練時從中隨機抽樣, 能打破資料的時間相關性, 避免「災難性遺忘」。

目標值 (Target) 計算: 用目標網路估計下一狀態的最高 Q 值, 再與環境回傳的獎勵組合成訓練目標。

實作結構導讀

DQN 訓練需要較多運算資源。在 Colab 中請開啟 GPU 加速, 以縮短 CartPole 範例的訓練時間。

講義進入尾聲, 最後我們整理常見的佈署問題與動手練習。

9. 疑難排解與自我檢查清單

強化學習的學習過程本身就是一場試錯。

常見錯誤與排除

檔案匯入失敗: 實作 GridWorld 前, 必須確認已將 `gridworld_env.py` 與 `agent.py` 上傳至 Colab 當前目錄, 否則 `import` 會報錯。

Colab 畫面崩潰: 請註解掉 `env.render()`。

訓練不收斂: 確認是否開啟 GPU, 並檢查 `epsilon_decay` 是否設定過快。

動手練習

行為實驗：試著將折扣因子 γ 從 0.9 改為 0.1，觀察 AI 是否變得極度短視，導致無法避開長遠的陷阱。

記憶測試：修改 `Replay Memory` 的長度，觀察記憶太短是否會讓 AI 出現「學了後面忘前面」的現象。

自我檢查清單

- ☐ 我能列舉出互動迴圈的五大元素 (S, A, R, S', Done) 嗎？
- ☐ 我理解為什麼 `CartPole` 在 12 度時會觸發 `done=True` 嗎？
- ☐ 我能解釋為什麼 DQN 的輸出層神經元數量必須與動作數一致嗎？
- ☐ 我知道 `deque` 相比 `list` 在處理經驗回放時的效能優勢嗎？
- ☐ 我能用白話解釋貝爾曼方程式（現在 = 獎勵 + 打折後的未來）嗎？